

Hafta 1 — Ortam Kurulumu, İlk Sınıf ve Nesne

Öğrenci Yönergesi — Adım Adım Uygulama

Bilgi	Değer
Ders	Nesne Tabanlı Programlama (2. Sınıf)
Ortam	Visual Studio Community 2026
Çalışma Zamanı	.NET 10 (Uzun Süreli Destek – LTS)
Dil	C#
Süre	Yaklaşık 2–3 saat
Teslim	Çalışan proje klasörü (.zip)

1. Bu Haftanın Hedefi

Bu ilk laboratuvarın amacı çok sayıda konu öğrenmek değil; geliştirme ortamını doğru kurmak ve nesne tabanlı programlamanın en temel iki kavramını — SINIF (class) ve NESNE (object) — sağlam biçimde kavramaktır. Bu haftayı iyi anladığınızda, dersin geri kalan tüm haftaları bu temelin üzerine kurulacaktır.

Föyü baştan sona sırayla takip edin. Kod parçalarını kopyalayıp yapıştırmak yerine kendiniz yazın; bu, sözdizimini (syntax) ve klavye alışkanlığını çok daha hızlı kazandırır.

2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- Visual Studio ile bir Konsol (Console) projesi oluşturmak,
- solution (çözüm), project (proje) ve namespace (ad alanı) kavramlarını açıklamak,
- class (sınıf), nesne, field (alan) ve method (metot) kavramlarını birbirinden ayırmak,
- kendi tanımladığınız bir sınıftan nesne üretip metodunu çağırmak,
- kullanıcıdan konsol üzerinden veri okuyup ekrana yazdırmak.

3. Kurulum: Visual Studio Community 2026

3.1. İndirme

- Tarayıcıdan visualstudio.microsoft.com adresine gidin.
- İndirme sayfasından Visual Studio Community 2026 sürümünü indirin. Bu sürüm öğrenciler ve eğitim amaçlı kullanım için tamamen ücretsizdir.
- İndirdiğiniz kurulum dosyasını (Visual Studio Installer) çalıştırın.

⚠ Dikkat: "Insiders" sürümünü seçmeyin

İndirme ekranında ayrıca "Insiders" adını taşıyan sürümler görebilirsiniz. Bunlar test/önizleme sürümleridir ve kararlı değildir.

Ders için normal (kararlı) Community 2026 sürümünü kullanın.

3.2. İş Yükünü (Workload) Seçme

Kurulum sihirbazı açıldığında sizden hangi bileşenleri yükleyeceğinizi seçmenizi ister. Bu ders için tek bir iş yükü yeterlidir:

1. Üstteki İş yükleri sekmesinde olduğunuzdan emin olun.
2. Aşağı inip Masaüstü ve Mobil başlığını bulun.
3. Bu başlığın altındaki ".NET masaüstü geliştirme" kutusunu işaretleyin. (Bu iş yükü C# ile hem konsol hem de Windows Forms uygulamaları geliştirmeyi kapsar.)
4. Diğer iş yüklerini (ASP.NET/web, Azure, Python, Node.js, .NET MAUI) işaretlemeyin — bu ders için gerekli değildir ve kurulumu gereksiz uzatır.
5. Sağ alttaki Yükle düğmesine basın ve kurulumun tamamlanmasını bekleyin.

İyi haber

Bu iş yükü, projelerimizi çalıştırmak için gereken .NET 10 SDK'sını da otomatik olarak birlikte getirir; ayrıca bir şey indirmenize gerek yoktur.

İleride başka bir bileşene ihtiyaç olursa Visual Studio Installer'ı tekrar açıp iş yükü ekleyebilirsiniz.

4. İlk Projeyi Oluşturma

Kurulum bittikten sonra Visual Studio'yu açın ve aşağıdaki adımları izleyin. (Menü adları Türkçe arayüze göre verilmiştir; parantez içinde İngilizce karşılıkları da yazılmıştır.)

1. Açılış ekranında Yeni proje oluştur (Create a new project) seçeneğine tıklayın.
2. Proje şablonları listesinden Konsol Uygulaması (Console App / C#) şablonunu seçin ve İleri (Next) deyin.
3. Proje adı olarak Hafta1_OgrenciKayit yazın, kaydedileceği konumu belirleyin ve İleri (Next) deyin.
4. Çerçeve (Framework) açılır listesinden ".NET 10.0 (Uzun süreli destek)" seçeneğini seçin.
5. "En üst düzey deyimleri kullanma" (Do not use top-level statements) kutusunu İŞARETLEYİN, ardından Oluştur (Create) deyin.

Neden bu kutuyu işaretliyoruz?

İşaretlenmezse Visual Studio, kodun içindeki class ve Main yapısını gizleyen sadeleştirilmiş bir şablon açar. Bu haftada tam da o yapıyı öğreneceğimiz için, gizli olmayan (klasik) hâlini görmek istiyoruz.

Kontrol Noktası ✓

Proje açıldığında ekranda Program.cs adlı bir dosya ve içinde bir miktar hazır kod görmelisiniz. Sağ tarafta ise Çözüm Gezgini (Solution Explorer) panelinde proje adınız listelenmiş olmalıdır.

4.1. Programı Çalıştırma: F5 ve Ctrl+F5

Projeyi çalıştırmanın iki yolu vardır: **F5** hata ayıklama (debug) ile çalıştırır; **Ctrl+F5** ise hata ayıklama olmadan çalıştırır ve konsol penceresi işiniz bitene kadar ekranda açık kalır. İlk denemelerinizde **Ctrl+F5** kullanmanız, "ekran açılıp hemen kapandı" sorununu önler.

GitHub Copilot önerisi hakkında

Community 2026, kod yazarken otomatik öneriler sunan GitHub Copilot ile birlikte gelir; bazen tüm satırı sizin yerinize yazmayı teklif edebilir.

Bu ilk haftalarda temeli gerçekten öğrenmek için önerileri kabul etmeden kodu kendiniz yazmaya çalışın.

5. Kodun Anatomisi

Proje oluşturulduğunda Program.cs dosyası aşağıdakine benzer bir iskeletle açılır. Bu haftanın konusunu anlamak için her satırın ne işe yaradığını bilmek önemlidir:

```
using System;
```

```
namespace Hafta1_OgrenciKayit
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Merhaba Nesne Tabanlı Programlama!");
        }
    }
}
```

Satır satır açıklama:

- `using System;` → Hazır kütüphaneleri projeye dâhil eder. Console sınıfı buradan gelir.
- `namespace` → Sınıfları gruplayan mantıksal bir klasör; isim çakışmalarını önler.
- `class Program` → Bir sınıf; kodumuzun içinde yaşadığı kalıp.
- `static void Main(...)` → Programın giriş noktası. Çalışma buradan başlar.
- `Console.WriteLine(...)` → Ekrana yazı yazdıran bir metod çağırısı.

⚠ Dikkat edilecek sözdizimi kuralları

Her komut satırı noktalı virgül (;) ile biter.

Kod blokları süslü parantez { } ile açılır ve kapanır; her açılan mutlaka kapanmalıdır.

C# büyük/küçük harfe duyarlıdır: `Console.WriteLine` çalışır, `console.writeline` çalışmaz.

6. Temel Kavram: Sınıf ve Nesne

En basit tanımıyla: **sınıf (class) bir kalıptır; nesne (object) o kalıptan üretilen somut örnektir.** Örneğin bir kurabiye kalıbı sınıftır; o kalıpla yapılan her bir kurabiye ise ayrı bir nesnedir. Tek bir kalıptan istediğiniz kadar kurabiye (nesne) üretebilirsiniz ve her birinin özellikleri birbirinden bağımsızdır.

Bir sınıfın içinde iki temel şey bulunur:

- **Field (alan):** Nesnenin verisi/özellliği. Örneğin öğrencinin adı, numarası, notu.
- **Method (metot):** Nesnenin yapabildiği iş. Örneğin bilgilerini ekrana yazdırmak veya ortalamasını hesaplamak.

Bir sınıftan yeni bir nesne üretmek için **new** anahtar sözcüğü kullanılır.

7. Uygulama: Öğrenci Kayıt Programı

Şimdi bir Öğrenci sınıfı tanımlayıp ondan nesneler üreteceğiz. Aşağıdaki adımları sırayla uygulayın.

Adım A — Öğrenci sınıfını ayrı bir dosyada oluşturun

Çözüm Gezgini (Solution Explorer) panelinde proje adına sağ tıklayın → Ekle (Add) → Sınıf... (Class...) → dosya adına Öğrenci.cs yazın. (Kural: her sınıf kendi dosyasında bulunur.) Ardından sınıfı şöyle doldurun:

```
namespace Hafta1_OgrenciKayit
{
    class Ogrenci
    {
        // Field'lar (nesnenin verisi)
        public string Ad;
        public string Soyad;
        public int Numara;
        public double Vize;
        public double Final;
    }
}
```

```
// Method (nesnenin davranışı)
public void BilgiYazdir()
{
    Console.WriteLine($"No: {Numara} | {Ad} {Soyad}");
    Console.WriteLine($"Vize: {Vize}, Final: {Final}");
}

public double OrtalamaHesapla()
{
    return Vize * 0.4 + Final * 0.6;
}
}
```

Adım B — Main içinde bir nesne üretip kullanın

Program.cs dosyasındaki Main metodunun içeriğini aşağıdaki gibi doldurun:

```
static void Main(string[] args)
{
    Ogrenci ogr1 = new Ogrenci();
    ogr1.Ad = "Ayşe";
    ogr1.Soyad = "Yılmaz";
    ogr1.Numara = 101;
    ogr1.Vize = 70;
    ogr1.Final = 85;

    ogr1.BilgiYazdir();
    Console.WriteLine($"Ortalama: {ogr1.OrtalamaHesapla()}");
}
```

Programı **Ctrl+F5** ile çalıştırın.

Ne görmelisiniz?

No: 101 | Ayşe Yılmaz

Vize: 70, Final: 85

Ortalama: 79

(Ortalama = $70 \times 0,4 + 85 \times 0,6 = 79$)

Adım C — Kullanıcıdan bilgi alın

Bilgileri kodun içine sabit yazmak yerine kullanıcıdan okuyalım. Aşağıdaki gibi Console.ReadLine kullanın:

```
Console.Write("Öğrenci adı: ");
ogr1.Ad = Console.ReadLine();

Console.Write("Numara: ");
ogr1.Numara = int.Parse(Console.ReadLine());
```

Neden int.Parse?

Console.ReadLine her zaman metin (string) döndürür. Numara bir sayı (int) olduğu için önce sayıya çevirmemiz gerekir.

int.Parse tam da bunu yapar: metni tam sayıya dönüştürür.

8. Kendi Göreviniz (Lab Teslimi)

Yukarıdaki adımları tamamladıktan sonra programı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

1. Aynı Ogrenci sınıfından ikinci bir nesne (ogr2) üretin ve farklı bilgilerle doldurun.
2. Her iki öğrencinin de bilgilerini ve ortalamasını ekrana yazdırın.
3. Programın başında ekrana bir başlık yazdırın (örn. "--- ÖĞRENCİ KAYIT SİSTEMİ ---").

4. Bilgilerin en az birini (örn. her iki öğrencinin adını) kullanıcıdan Console.ReadLine ile alın.

İki nesne, tek kalıp

ogr1 ve ogr2 aynı Ogrenci sınıfından üretilir ama verileri birbirinden tamamen bağımsızdır.

Bu, sınıf–nesne ilişkisinin en önemli fikridir: bir kalıptan çok sayıda bağımsız nesne.

9. Sık Yapılan Hatalar

- **new unutmak:** Ogrenci ogr1; yazıp new Ogrenci() dememek NullReferenceException hatasına yol açar. "Kalıbı tanıttın ama nesneyi üretmedin" demektir.
- **Noktalı virgül veya süslü parantez eksikliği:** Derleyici hata verir. Alttaki Hata Listesi (Error List) panelini okumayı alışkanlık edin.
- **Büyük/küçük harf:** Console (doğru) ile console (yanlış) aynı şey değildir.
- **Dosya adı ile sınıf adı farkı:** Zorunlu değildir ama aynı tutmak (Ogrenci.cs içinde Ogrenci) karışıklığı önler.
- **Ekran hemen kapanıyor:** Programı F5 yerine Ctrl+F5 ile çalıştırın; konsol penceresi açık kalır.

10. Değerlendirme

Bu laboratuvar aşağıdaki ölçütlere göre değerlendirilecektir:

Ölçüt	Puan
Proje doğru oluşturulmuş ve hatasız derleniyor	20
Ogrenci sınıfı uygun field ve method'larla tanımlanmış	25
En az iki nesne üretilmiş ve doğru kullanılmış	20
Kullanıcıdan giriş alma çalışıyor	20
Kod okunabilir (isimlendirme, girinti, düzen)	15
TOPLAM	100

11. Ödev (Gelecek Haftaya Hazırlık)

Üç öğrenciyi üç ayrı Ogrenci nesnesi olarak oluşturun ve aralarından en yüksek ortalamaya sahip olanı bulup ekrana yazdırın.

Bunu yaparken "üç ayrı değişkeni tek tek yönetmek hantal" hissini yaşayacaksınız. **Bu his tesadüf değil:** ileride öğreneceğimiz koleksiyonlar (List) tam da bu sorunu çözmek için vardır. Ayrıca field'ların herkes tarafından serbestçe değiştirilebilmesi (örneğin negatif not girilebilmesi) bir sonraki hafta göreceğimiz kapsülleme (encapsulation) konusuna güzel bir giriş olacaktır.